Carbon · Window, pages and panels

GUI audit and stress test

Bugs, rendering errors, performance, missing features and crash resistance in everything around the calculator, with fixes and a stress-test suite.

Date	2 November 2026
Code audited	Commit `1981cd3` on `main`, plus your uncommitted unsaved-dot change (`gui.py`, `regneark.html`)
Scope	`src/gui.py` (window, header, drawer, search, pages, all 14 panels, Regneark tabs and files, molecule page, settings, changelog, license dialog), `Assets_math_live/mathfield.html`, the GUI parts of `regneark.html`, `license_manager.py`, `build.spec`
Environment	Windows 11, Python 3.13, PySide6 6.11.2, headless (`QT_QPA_PLATFORM=offscreen`, software Qt Quick renderer). Settings went to a temporary file, not your registry. Nothing was opened on your desktop.
Source changes	None. One new test file: `tests/test_stress_window.py`.

Summary

I found **34 issues**: 0 critical, 4 high, 12 medium, 16 low and 2 notes. Every one was reproduced headless, and the measurements are given with each finding. The calculation engine isn't repeated here. It has its own report (`Interpreter_Audit_Report.pdf`), and the IDs below continue its numbering.

The good news first: the GUI is hard to crash. Across 1 500 random page switches, 1 000 drawer toggles, window sizes from 1 × 1 to 6 000 × 4 000 pixels, 100 000-character searches, 60 open tabs, 40 open/close cycles and about 60 hostile inputs in all 14 panels, there wasn't a single crash, hang or leaked process.

The real problems are elsewhere. Every input box is its own browser process (1.6 GB with all panels open), errors inside callbacks vanish and resurface in unrelated code, two help texts tell students to type the numbers the wrong way round, and the installer leaves out the molecule editor. After that come the rough edges a student will notice: answers that shrink to 7 px, close prompts that ask even when everything is saved, Enter meaning "discard", and Danish decimal commas that quietly change the result.

N-not that I enjoyed clicking your buttons ten thousand times. But most of the fixes are one-liners, and the ten at the top take an afternoon.

Severity	Count	Meaning
Critical	0	Crash, permanent freeze, security hole, or wrong answers in everyday use
High	4	Silent failures, data loss risk, wrong answers in specific cases, severe slowdowns
Medium	12	Broken feature with a workaround, misleading output, noticeable slowness
Low	16	Rare edge cases, cosmetics, clean-up
Info	2	Notes on unfinished work

Fix these ten first

ID	What to do
P-06	Add `--renderer-process-limit=4` to the Chromium flags: 1.6 GB becomes 0.97 GB. One line.
M-06	Add `Assets_Ketcher` to `datas` in `build.spec`, so the installed app has its molecule editor again. One line.
B-26	Swap the two sentences in the Volumenprocent and Masseprocent help texts, and fix the examples.
U-01 + U-02	Use `view.property("dirty")` in both close prompts, and make "Annuller" the default button.
B-27	Return early in `remove_interpreter_tab` when the view is already gone. Two lines.

ID	What to do
C-07	Catch errors inside the save, network and changelog callbacks, and add a global error handler that writes a log file.
C-08	Parse the stored pK tables before clearing them, and fall back to the defaults.
R-10 + R-11 + R-14	Show text results in a word-wrapping label, keep units on one line, and render errors with `latex_text`.
E-04 + B-28	Detect decimal commas in panel boxes and say "Brug punktum som decimaltegn".
R-12, R-13, R-15	Three one-line fixes: the font name, the `->` shortcut and `setAutoRaise(True)`.

1 Scope and method

- **Reading.** All 3 116 lines of `gui.py`, `mathfield.html`, the GUI parts of `regneark.html`, `license_manager.py`, `path.py` and `build.spec`, plus a pyflakes pass.
- **Probing.** A headless harness built the real `MainWindow` and drove it with its own methods and with QTest mouse and key events. Every exception that escaped a slot or a thread was caught (`sys.excepthook` and `threading.excepthook`). Memory was measured with psutil as unique set size, so memory shared between Chromium processes isn't counted twice.
- **Screenshots.** Every page, panel and dialog was captured with `QWidget.grab()` at 100 %, 150 % and 200 % scaling, web views included, and checked by eye.
- **Stress.** Far past normal use: 1 500 random page switches plus 20 000 without animation, 1 000 drawer toggles, 1 000 toggle-switch clicks, sizes from 1 × 1 to 6 000 × 4 000, a 100 000-character search, 60 tabs, 2 000-character tab names, 40 open/close cycles, every panel open at once and a 300-entry pK table.
- **Fault injection.** Read-only files, missing folders, no network, a local server that answers every request with a school-Wi-Fi login page, empty and non-UTF-8 replies, a damaged settings value, a frozen tab, closing tabs mid-calculation, double clicks and a slow license server.
- **Isolation.** Everything ran on a copy of the project in my scratch folder. No window opened on your desktop and your registry wasn't touched.
- **Not covered.** A headless run can't reach the native Windows frame code (`nativeEvent` hit-testing, Aero snap, moving between monitors with different scaling), real GPU rendering or the installed PyInstaller build. M-06 is based on `build.spec` and your failing test, not on an installed copy.

Line numbers refer to the working tree on 2 November 2026 (commit `1981cd3` plus your uncommitted unsaved-dot change).

2 Findings

IDs continue from the interpreter report: **C** crash, freeze or silent failure · **B** bug or wrong result · **R** rendering · **P** performance · **M** missing or incomplete · **U** usability (new) · **E** messages and text · **Q** code quality · **WIP** unfinished work. Test names in `tests/test_stress_window.py` carry the same IDs.

High (4)

HIGH P-06 Every input and output box is a separate browser process

Where `src/gui.py:787` (`MathInput`), `:838` (`MathOutput`), `:1920` (`PanelPage.show_panel`)

What happens Each box in a panel is a complete `QWebEngineView` running `mathfield.html`, and Chromium gives each one its own renderer process. With every panel type open there are 55 web views, 55 child processes and **1 601 MB** of memory. Closing a panel only hides it, so nothing is freed until Carbon exits. Each Regneark tab adds another process (60 tabs: 61 processes). On an 8 GB school laptop that is a fifth of the memory before the student has typed anything. Measured (unique memory): start 486 MB / 2 processes, all panels 1 601 MB / 55, plus 20 tabs 2 008 MB / 75.

Fix Cap the renderer processes where the Chromium flags are set (and see Q-02 about that line). Measured with the cap: **971 MB / 4 processes** with all panels open. `--process-per-site` goes further (797 MB / 1 process), but then one frozen page (C-01) freezes every page, so I'd use the cap. Long term: use plain `QLineEdit`/`QLabel` for boxes that never show maths (names, pKs values, DNA strings, most results), use one page per panel instead of one per box, and free panels when they are closed.

```
os.environ["QTWEBENGINE_CHROMIUM_FLAGS"] = "--log-level=3 --renderer-process-limit=4"
```

HIGH C-07 Errors inside callbacks vanish and resurface in unrelated code

Where The `runJavaScript` and network callbacks in `src/gui.py`: `save_regneark.write` (`:2977`), `close_interpreter_tab` (`:2776`), `draw_smiles` (`:3005`), `show_name` (`:3025`), `show_changelog` (`:2072`), `set_dirty` (`:2769`), and the worker thread in `_InterpreterBridge.run` (`:759`)

What happens When Python code in one of these callbacks raises, PySide6 doesn't report it there. The error stays pending and appears at the next Python call, usually as "Error calling Python override of QMainWindow::event()", and that event is lost. In one run the pending error landed in the web channel's call to `help()` ("Slot '_InterpreterBridge::help()' not found"), so that new tab never got its help tooltips. The installed app has no console (`console=False`), so nothing is shown or logged at all. Reproduced triggers: saving to a read-only file or a missing folder (nothing is written and no message appears, only the dot stays), Ctrl+S before a new tab has loaded, an empty PubChem reply (`IndexError`), a login page instead of PubChem (`JSONDecodeError`), a changelog that isn't UTF-8 (`UnicodeDecodeError`), closing a tab during a calculation (`RuntimeError: Signal source has been deleted` in the worker thread) and a dirty signal from a closed tab (`AttributeError`).

Fix Catch errors inside each callback and show a Danish message. Install a last-resort handler at start-up (`sys.excepthook` and `threading.excepthook`) that appends to a log file in `CONFIG_DIR` and shows a short message. In the worker thread, catch `RuntimeError` around the emit. For saving:

```
def write(data):
    try:
        text = json.dumps({"rows": json.loads(data)}, ensure_ascii=False, indent=2)
        Path(path).write_text(text, encoding="utf-8")
    except (OSError, TypeError, ValueError) as e:
        QMessageBox.warning(self, "Gem regneark", f"Filen kunne ikke gemmes:\n{e}")
        return
    view.page().runJavaScript(f"markSaved({json.dumps(data)})")
```

HIGH B-26 Two help texts tell students to type the numbers in the wrong boxes

Where `src/gui.py:239-246` (`HELPTEXT["volumenprocent"]`, `HELPTEXT["masseprocent"]`) against the panels at `:1427` and `:1459`

What happens Both help texts say "Indtast samlet … i det første input og specifik … i det andet", but in both panels the first box is "Specifik[Enhed]" and the second is "Samlet[Enhed]", and the engine divides the first by the second. Following the help gives the inverted ratio: the help's own example, 4[kg] then 300[g], gives **1000 %** instead of 7.5 %. The examples are odd as well. In Volumenprocent the specific volume (300 dl = 30 l) is larger than the total (4 l), and the Udbytteprocent example (practical 4 kg, theoretical 300 g) also gives 1000 %.

Fix Swap the sentences so they match the boxes ("Indtast specifik masse i det første input og samlet masse i det andet") and use examples that make sense, for example 300[g] and 4[kg] (7.5 %).

HIGH M-06 The installed app has no molecule editor

Where `build.spec:31-35` (`datas`)

What happens `datas` bundles `Assets`, `config.json` and `Assets_math_live`, but not `Assets_Ketcher`. In the frozen app, `resource_path("Assets_Ketcher/index.html")` points to a file that doesn't exist, so the Molekyle Tegner page (header button and drawer entry) stays empty and "Tegn"/"Find navn" can't work. `test_path.py::test_build_bundles_resource[Assets_Ketcher]` fails for exactly this reason. The previous report only mentioned it in a table, so it gets an ID now.

Fix Add the folder to `datas`. It adds about 31 MB to the app folder.

```
(str(PROJECT_ROOT / "Assets_Ketcher"), "Assets_Ketcher"),
```

Medium (12)

MEDIUM U-01 The close prompts ask even when everything is saved

Where `src/gui.py:2776` (`close_interpreter_tab`) and `:3092` (`closeEvent`)

What happens Both prompts ask JavaScript whether any line is non-empty, not whether the tab has unsaved changes. A file you just opened or saved (no dot) still says "Det, der ikke er gemt, går tabt" when you close the tab or Carbon. Prompts that appear for no reason teach people to click "Luk" without reading, which is exactly when they lose real work.

Fix Use the dirty flag you track now. That also removes the JavaScript round trip that lets a frozen tab block closing (M-04).

```
def close_interpreter_tab(self, index):
    view = self.interpreter_stack.widget(index)
    self.remove_interpreter_tab(view, bool(view.property("dirty")))

def closeEvent(self, event):
    views = [self.interpreter_stack.widget(i) for i in range(self.interpreter_stack.count())]
    text = "Vil du lukke Carbon?\nDet, der ikke er gemt, går tabt."
    if any(v.property("dirty") for v in views) and not self.ask_close("Luk Carbon", text):
        event.ignore()
        return
    super().closeEvent(event)
```

MEDIUM U-02 Enter in "Vil du lukke?" throws the work away

Where `src/gui.py:2778-2792` (`ask_close`)

What happens "Luk" is added first, so it gets the focus and becomes the default button. Pressing Enter, the most common reflex in a dialog, closes and discards unsaved work. Checked at 60, 150, 400 and 900 ms after the dialog opened: focus and default were always "Luk", and `ask_close` returned `True`.

Fix Make the safe choice the default: `msg.setDefaultButton(msg.addButton("Annuller", QMessageBox.RejectRole))`.

MEDIUM B-27 Double-clicking a tab's × opens a new empty tab

Where `src/gui.py:2774-2803` (`close_interpreter_tab`, `remove_interpreter_tab`)

What happens Each click sends its own question to JavaScript, so a double click calls `remove_interpreter_tab` twice for the same view. With two tabs open, the second call sees one tab left and creates a new one before noticing that its view is already gone: closing "Ny Fil 2" leaves a fresh, empty "Ny Fil 2" (3 out of 3 runs). If the sheet has content, the second call either shows the close dialog again or raises `AttributeError` in `tab_name(-1)` (C-07).

Fix Return early when the view is already gone:

```
def remove_interpreter_tab(self, view, used):
    if self.interpreter_stack.indexOf(view) < 0:
        return
```

MEDIUM R-10 Long results shrink to 7 px and are cut off

Where `Assets_math_live/mathfield.html:101-124` (`fit`, `MIN_SIZE = 7`), `src/gui.py:841` (`MathOutput` is fixed at 27 px)

What happens A `MathOutput` is a single 27 px line. `fit()` shrinks the font until the text fits, but stops at 7 px, and the rest is clipped. A 60-codon DNA translation ends up as 1 014 px of 7 px text in a 266 px box: three quarters of the answer can't be seen, and the visible part is too small to read. Long Afstem results and long error messages behave the same way (figure 1).

Fix Show text results (peptide chains, balanced reactions, messages) in a word-wrapping `QLabel` or the existing `TextOutput`, and keep `MathOutput` for short formulas. If you keep MathLive here, raise `MIN_SIZE` to about 10 and let the field scroll sideways.



Figure 1. DNA Translation with ATG × 60. The input and the result are shrunk to 7 px and cut off at the right edge.

MEDIUM R-11 Units with a fraction are clipped

Where `src/gui.py:650` (the fraction rule in `calc_to_latex`), used by `MathOutput._push` (`:860`)

What happens `calc_to_latex` turns every a/b into `\frac{a}{b}`, so a unit result such as 0.0025 mol/ml is drawn as a stacked fraction in the 27 px box. `fit()` only compares the height, not the position, so the fraction keeps its 12 px size and hangs below the box. Measured: content bottom 27.8 px, box bottom 27 px, so the denominator is cut off (figure 2, left).

Fix Keep units on one line in panel outputs (give `calc_to_latex` a flag that skips the fraction rule), or make the output about 40 px high when the LaTeX contains `\frac`.

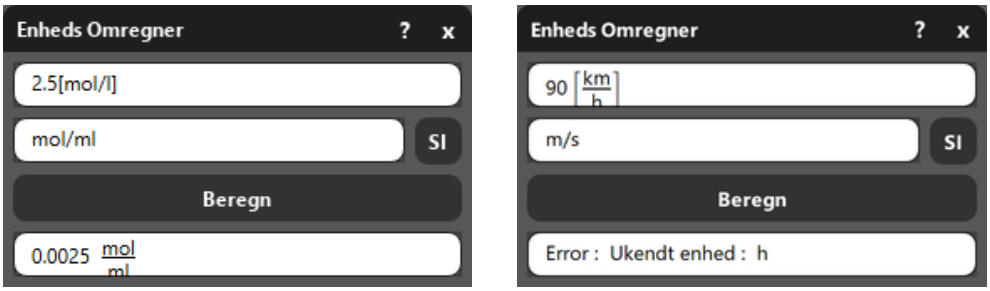


Figure 2. Left: 2.5[mol/l] in mol/ml, with the denominator of the unit cut off (R-11). Right: an unknown unit, shown as "Error : Ukendt enhed : h" with the spacing of a maths relation (R-14).

MEDIUM U-03 Panel results lose their labels

Where `src/gui.py:1184-1348` (pH, pOH and Analyse panels), `:1643-1678` (H3O+ og OH-)

What happens The only label on an output box is its placeholder, and it disappears as soon as a result is shown. Analyse then shows five unlabelled values (CH3, 15.04, 0.5, 4), and the molar mass has no unit. The pH and pOH panels show two bare numbers, and because the pOH panel puts pOH first and pH second, the same two numbers mean opposite things in two neighbouring panels (figure 3).

Fix Add a small caption to each output box, or put the name in the value ("pH = 2.9", "M = 180.16 g/mol").



Figure 3. Analyse of CHHH and pH of pKs −2 at 1 mol/l. Once the results are in, nothing says what they are.

MEDIUM B-28 "4,76" in the Bjerrum panel silently becomes two acids

Where `src/gui.py:1374` (`bjerrum_panel`), `HELPTEXT["bjerrum"]` (`:211`)

What happens Commas separate pKs values, and Danish students write decimals with a comma. "4,76" draws the legend "pKs: 4.0" and "pKs: 76.0". The 76 curve lies outside the 0-14 axis, so the student sees a single, wrong curve at pH 4 and no warning (figure 4).

Fix Separate values with ";" or spaces, and accept a comma as the decimal sign ("4,76" means 4.76). Warn when a pKs is outside about −2 to 16.

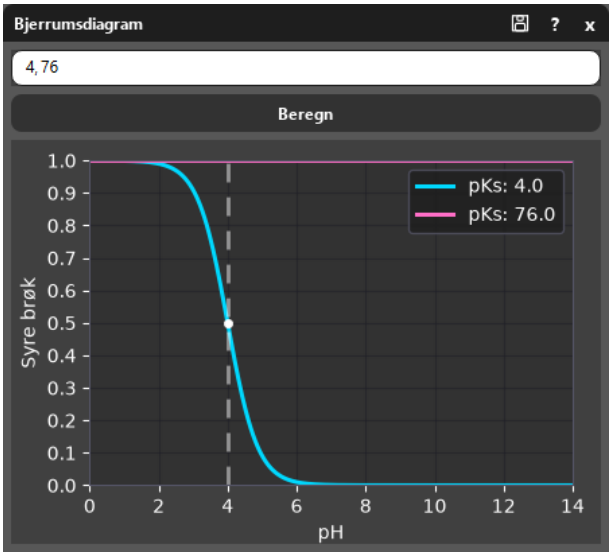


Figure 4. Bjerrumdiagram for "4,76": one wrong curve at pKs 4, and a legend entry for an invisible curve at pKs 76.

MEDIUM E-04 Panel errors show calculator syntax and Python messages

Where The `calc()` functions in `src/gui.py:1171-1678`, which build `Funktion(a,b)` strings

What happens The panels build a Regneark call from the boxes, so the error describes the call, not what the student typed. "1,5[m]" to "cm" gives `Error: EnhedKonvertering(Quantity,ny Enhed eller "SI" for SI enhed)`. pH with concentration 0 gives "math domain error", Puffer with 0 acid gives "float division by zero", Mængdeberegning with "abc" gives "could not convert string to float: 'abc'", and pH 1e400 gives "cannot convert float infinity to integer". A decimal comma in any box changes the number of arguments and gives the generic syntax error, with no hint that a dot is expected, although that is the most likely mistake.

Fix Check the boxes before calling the engine. If a box contains a comma, say "Brug punktum som decimaltegn (1.5, ikke 1,5)". Translate the common Python errors per panel ("Koncentrationen skal være større end 0"). The engine-side list is E-01 in the interpreter report.

MEDIUM P-07 Panel calculations freeze the window

Where `src/gui.py:1334` (Analyse) and every panel `calc()`

What happens Panel calculations run inside the button handler on the GUI thread. Analyse takes 0.7-1.5 s on every click (C6H12O6 1.42 s, even "H" 0.70 s), and during that time the window neither repaints nor reacts: a 50 ms timer got 0 ticks. The cost is mendeleev: every `element()` lookup takes 0.14-0.21 s, and nothing is cached. Big Afstem or Bjerrum inputs block in the same way.

Fix Cache the few element values Analyse needs (a dictionary filled on first use makes repeated calls instant), run panel calculations in a worker (`QThreadPool`, or the killable process from C-03) and show a busy cursor.

MEDIUM P-08 Carbon takes about 5 s to start

Where `src/gui.py:1-102` (imports), `src/chem.py:2-15`, `src/gui.py:1975` (`create_molecule_page`)

What happens `import gui` alone takes **3.8 s**, and 3.5 s of that is `chem`: `quantity` (pandas and requests) 1.3 s, mendeleev 1.2 s, `matplotlib.pyplot` 0.5 s and `scipy.linalg` 0.4 s. Building `MainWindow` takes another 1.2 s, and the molecule page loads Ketcher's 29.6 MB JavaScript bundle at every start, even when it is never opened. All in all, a student waits about 5 s before the window appears.

Fix Import the heavy modules where they are used (mendeleev in the Analyse code, scipy in Afstem, pyplot only for the export window), create the molecule page the first time it is opened, and show a `QSplashScreen` while importing.

MEDIUM M-07 The molecule page hides network problems

Where `src/gui.py:2995-3034` (`name_to_structure`, `draw_smiles`, `structure_to_name`, `show_name`)

What happens Offline, "Tegn" shows "Ikke fundet", which says the molecule doesn't exist when the network is the problem. Behind a school-Wi-Fi login page (simulated with a local server that answers everything with an HTML page), "Tegn" passes "<!DOCTYPE html>" to Ketcher as SMILES and shows nothing, and "Find navn" raises `JSONDecodeError` (C-07). An empty reply raises `IndexError`. "Find navn" on an empty canvas does nothing. There is no "Søger..." state and no time-out, so a slow connection looks like a broken button.

Fix Treat network errors separately ("Ingen forbindelse til PubChem"), check the content before using it (one line, no "<"), show "Søger..." while the request runs and "Tegn et molekyle først" for an empty canvas, and set `request.setTransferTimeout(10000)`.

MEDIUM U-04 The license dialog freezes during activation and loses the key

Where `src/gui.py:981-999` (`LicenseWindow.try_accept`), `src/license_manager.py:10-19` (`activate`, `timeout=10`)

What happens `activate()` calls `requests.post` on the GUI thread with a 10 s time-out, and the dialog doesn't react meanwhile (with a simulated slow server, a 50 ms timer got 0 ticks in 3 s). On a student's very first start, Windows may show it as "Not responding". Errors are written into the key field itself: an invalid key is replaced by "Ugyldig licens", so the student must paste it again, and pressing Ok once more validates the message text.

Fix Show messages in a separate `QLabel` under the field and keep the key. Activate in a `QThread` (or with `QNetworkAccessManager`), disable the buttons and show "Aktiverer...".

Low (16)

LOW R-12 The Regneark's suggestion list and help tooltip use the wrong font

Where `Assets_math_live/regneark.html:47` and `:58`

What happens Both rules ask for `font: 14px "Segoe-UI"`. The font is called "Segoe UI" with a space, so Chromium falls back to its default, and the autocomplete list looks different from the rest of Carbon (figure 5).

Fix `font: 14px "Segoe UI", sans-serif;` in both places.



Figure 5. The suggestion list after typing "L", in the fallback font.

LOW R-13 "->" isn't turned into an arrow in panel boxes

Where `Assets_math_live/mathfield.html:79`

What happens The panel boxes replace MathLive's default shortcuts with only *, ** and <->, so typing H2O->H2+O2 in Afstemning shows "H2O− > H2 + O2", while the Regneark shows an arrow. The calculation still works, it just looks wrong (figure 6).

Fix Add `"->": "\\to"` to `inlineShortcuts`. `Latex_to_calc` already turns `\to` back into `->`.

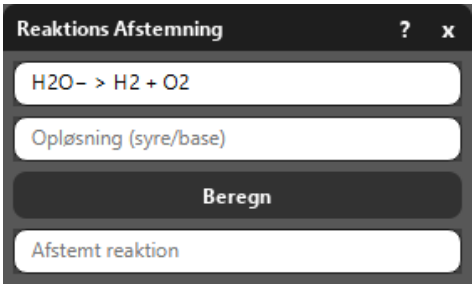


Figure 6. H2O->H2+O2 typed key by key into the Afstemning panel.

LOW R-14 Errors in panels render as "Error :"

Where `src/gui.py:861` (`MathOutput._push`)

What happens Panel errors go through `calc_to_latex` and end up inside `\mathrm{…}`, which is maths mode, so every colon gets the spacing of a relation sign ("Error : Ukendt enhed : h", figure 2, right). The Regneark avoids this by using `latex_text` for errors.

Fix In `_push`, use `latex_text(self._raw)` when the text starts with "Error".

LOW R-15 The Save button never shows its hover icon

Where `src/gui.py:2631` (`file_btn`)

What happens `hover_icon()` relies on `QIcon.Active`, which Qt only uses for auto-raised tool buttons. The header buttons are auto-raised (121 pixels change on hover), but `file_btn` isn't, so `Save_Hover.svg` is never shown (0 pixels change).

Fix `file_btn.setAutoRaise(True)`.

LOW R-16 The dropdown arrows point the wrong way

Where `src/gui.py:1777` (`Dropdown.toggle`), `:2150-2156` (drawer), `:2853-2854` ("Sprog" box)

What happens Closed sections show an arrow pointing up and open sections an arrow pointing down, in the sidebar, the drawer and the "Sprog" box. The usual conventions are right/down (disclosure) or down/up (drop-down list), so the closed state reads as "already open".

Fix Swap the two images in `Dropdown.toggle`, the drawer icon and the combobox style.

LOW R-17 Tall panels are cut off in small windows

Where `src/gui.py:1121-1127` (`_clamp_to_parent`), `:1920-1932` (`show_panel`)

What happens Panels can't scroll or be resized. When the page is shorter than a panel (Analyse 464 px, Bjerrum 407 px, Mængdeberegning 384 px), the bottom is cut off, often including the results. At 600 × 450, three of the twelve Kemi panels stuck out of the page (figure 7). New panels are placed with a 20 px cascade and aren't clamped.

Fix Put the panel body in a `QScrollArea`, or limit the panel height to the canvas height, and clamp new panels too.



Figure 7. All Kemi panels at 600 × 450. Mængdeberegning ends below the window.

LOW R-18 The version label covers page content

Where `src/gui.py:2011-2013`, `:2257-2261`

What happens "Bv 2.0" is a child of the main window placed over the bottom-right corner of every page. It sits on top of the Regneark's scrollbar corner, Ketcher's canvas and any panel dragged into that corner.

Fix Show the version on the main menu page, or in the drawer next to "Nyheder".

LOW U-05 Clicks during the page animation are thrown away

Where `src/gui.py:2915-2918` (`slide_to`)

What happens `slide_to` ignores every request while the 200 ms animation runs. In the stress run 1 212 of 1 500 rapid clicks were dropped. A user who clicks Kemi and then Fysik quickly stays on Kemi, while the header underline jumps to Fysik and back.

Fix Remember the last requested page and slide to it when the running animation finishes.

LOW U-06 Missing shortcuts and keyboard access

Where `regneark.html:600` (only Ctrl+Z), `src/gui.py:2630` (only Ctrl+S), `:1886-1888`, `:2634`, `:2648`, `:2659` (`Qt.NoFocus`), `:3061-3069` (F11/Esc)

What happens There is undo but no redo (Ctrl+Y). Open, Gem som, new tab, close tab and switching tabs have no shortcuts (Ctrl+O, Ctrl+Shift+S, Ctrl+T, Ctrl+W, Ctrl+Tab). Esc doesn't cancel renaming a tab. The sidebar buttons, the mode switch, the file button, the tab bar and "+" all use `Qt.NoFocus`, so they can't be reached with Tab. Leaving full screen with F11 gives a maximized window, but with Esc a normal one.

Fix Add `QShortcuts` next to the Ctrl+S one, keep a redo stack next to `undoStack`, restore `self.renaming` on Esc, and use the same window state for F11 and Esc.

LOW B-29 The same file can be open in two tabs

Where `src/gui.py:3053-3059` (`load_regneark`)

What happens Opening a .crbn that is already open creates a second tab with the same path. Saving either tab silently overwrites the other one's changes.

Fix Before opening, look for a tab whose `property("path")` is the same file, and switch to it instead.

LOW B-30 Problems in the pKs and pKb tables

Where `src/chem.py:205-224` (defaults), `:352-370` (lookup), `src/gui.py:407-492` (`pk_table`)

What happens Names are matched case-sensitively, so "Testsyre" doesn't find "testsyre". The defaults contain "Salpetersyre" with a capital S (so "salpetersyre" fails) and the misspelt duplicate "svolsyre" next to "svovlsyre". Acetic acid is only there as "ethansyre", not "eddikesyre". The pKb table is empty, although the pOH help says base names can be used. The table accepts "nan" and "inf" as values. A deleted entry can't be restored, and a saved table hides new defaults in later versions for good.

Fix Store and look up names in lower case, fix the data, add the common names, validate with `math.isfinite`, add a "Nulstil" button, and merge saved entries over the defaults instead of replacing them.

LOW P-09 The pK table gets slower with every entry

Where `src/gui.py:438-460` (`refresh`)

What happens Every add or delete rebuilds all rows, each with its own widget, layout, label and button. One add costs 81 ms at 50 entries, 143 ms at 100, 427 ms at 200 and 871 ms at 300, so the cost grows with the square of the size. Normal tables (about 20 entries) are fine.

Fix Insert or remove only the affected row, or use a two-column `QTableWidget`.

LOW C-08 A damaged pK table in the settings stops Carbon at start-up

Where `src/gui.py:277-281` (`load_dict_from_settings`), `:1950-1951`

What happens If the stored `pKa_values` or `pKb_values` isn't valid JSON, `json.loads` raises inside `MainWindow.__init__` and Carbon doesn't start, with no message. The table is also cleared before parsing, so a failed load would leave it empty. The only way out is the registry editor.

Fix Parse first, replace afterwards, and keep the defaults on any error:

```
def load_dict_from_settings(key, target):
    try:
        data = json.loads(_settings.value(key) or "null")
    except (TypeError, ValueError):
        return
    if isinstance(data, dict):
        target.clear()
        target.update(data)
```

LOW M-08 The changelog misbehaves offline and behind login pages

Where `src/gui.py:2068-2084`

What happens Clicking "Nyheder" offline does nothing at all. Behind a school-Wi-Fi login page, the login page is shown as the news, and closing it marks the real changelog as read for good. A reply that isn't UTF-8 raises `UnicodeDecodeError` (C-07). Esc doesn't close the overlay, even though it grabs the keyboard.

Fix Show "Kunne ikke hente nyheder" on errors, only show and mark as read when the text looks like your changelog (for example it starts with "#"), and close on Esc.

LOW E-05 Danish UI text

Where Panel titles, placeholders and help texts in `src/gui.py`, `regneark.html:975-977`, the Bjerrum axis label in `src/chem.py:518`

What happens Several compound words are written apart: "Enheds Omregner" (Enhedsomregner), "Reaktions Afstemning" (Reaktionsafstemning), "Molekyle Tegner" (Molekyletegner) and "Syre brøk" (syrebrøk). Other slips: "skrivt" (skrift) in `SUB_HELP`, the placeholders "Pks"/"Pkb" (pKs/pKb), "syrens navn for nogle baser" (basens navn) in the pOH help, "ctrl+Enter" (Ctrl+Enter) in the Regneark hint, "Ny Fil" (Ny fil), "Bjerrumdiagram" in the menu but "Bjerrumsdiagram" as the panel title, and "codon" (kodon) and the English "Error" in messages.

Fix Fix the strings, and recheck the help examples while you are there (B-26).

LOW Q-02 Leftovers in gui.py

Where `src/gui.py`

What happens pyflakes reports 18 unused or duplicate imports (`QGraphicsView`, `QGraphicsScene`, `QGraphicsEllipseItem`, `QGraphicsRectItem`, `QColorDialog`, `QCompleter`, `QMimeData`, `QRectF`, `QPixmap`, `QTextCursor`, `QDrag`, `QBrush`, `QPen`, `QPalette`, `dataclasses.field`, a second `Signal`, `chem` and `biotek` twice), and `field` is shadowed in two loops. `træ = biotek.graph()` (`:1004`) is never used. `export_btn` is a `QToolButton`, but its style sheet targets `QPushButton` (`:1051`). The Chromium flags are set twice: line 18 sets `--disable-gpu`, and line 3095 replaces it with `--log-level=3`, so the app runs with the GPU while the tests run without it. "Afbryd" in the license dialog calls `sys.exit(0)` inside a slot instead of `self.reject()`.

Fix Remove the unused imports and the dead global, set the Chromium flags in one place (decide whether you want `--disable-gpu` at all), and use `self.reject()` for "Afbryd".

Info (2)

INFO WIP-02 Section mode breaks the mode switch

Where `regneark.html:774` (`addSection` inside `plotElement`), `:913-923` (`setMode`), `src/gui.py:1729-1734` (`SegmentedControl.setCurrent`), `:2836`

What happens `addSection` is still declared inside `plotElement`, so it can't be called (`typeof addSection` is "undefined", WIP-01). Once a line is in "section" mode, `reportMode` sends "section" and `setCurrent` raises `KeyError`. The error comes back on every tab switch, because `show_interpreter_tab` asks for the mode of the last field.

Fix Move `addSection` to the top level of the script, and ignore unknown modes: `if value not in self.buttons: return`.

INFO I-01 Unfinished areas

Where Several places

What happens The Fysik page only has the unit converter. The main menu page is only the logo, with no shortcuts or recent files. The "Sprog" box has one entry and isn't connected to anything. There is no single-instance check, so double-clicking a .crbn while Carbon is open starts a second full copy (about 0.5 GB more) instead of opening a tab. The Bjerrum export opens a separate matplotlib window with an English toolbar, and `chem.bjerrum` changes matplotlib's global style for the rest of the session.

Fix Nothing urgent. Worth a place on the to-do list.

---

3 Stress-test results

OK means it worked within a reasonable time. **SLOW** means it worked, but a user would notice. **FAIL** means a crash, hang, freeze, wrong output, lost data or missing feedback.

Window and navigation

Case	Size	Result	Verdict	Note
Start-up	import + window	3.8 s + 1.2 s	SLOW	P-08
Random page switches	1 500 clicks	no errors, header in sync	OK	1 212 clicks dropped (U-05)
Page switches without animation	20 000	16 s, history 20 068 entries	OK	
Drawer toggles	1 000	consistent end state	OK	
Window size	1 × 1 to 6 000 × 4 000	minimum 600 × 196 kept	OK	with the drawer open
Search	100 000 characters, emoji, RTL	0.2 s	OK	
Panel dragged off the page	4 000 px	stays inside	OK	
All Kemi panels in a small window	600 × 450	3 of 12 cut off	FAIL	R-17
Display scaling	150 % and 200 %	sharp icons and text	OK	

Panels

Case	Size	Result	Verdict	Note
Every panel, hostile inputs	14 panels, about 60 inputs	0 exceptions	OK	messages are poor (E-04)
Every panel open	55 web views	1 601 MB, 55 processes	FAIL	P-06
Same, `--renderer-process-limit=4`	55 web views	971 MB, 4 processes	OK	the suggested fix
Same, `--process-per-site`	55 web views	797 MB, 1 process	OK	one freeze freezes all
Analyse	per click	0.7-1.5 s, window frozen	SLOW	P-07
DNA translation	60 codons	7 px, three quarters cut off	FAIL	R-10
Unit result with a fraction	mol/ml	denominator clipped	FAIL	R-11
Bjerrum "4,76"		two curves, pKs 4 and 76	FAIL	B-28

Tabs and files

Case	Size	Result	Verdict	Note
Open tabs	60	5.7 s, 61 processes, "+" reachable	SLOW	P-06
Tab name	2 000 characters	window width unchanged	OK	
Open and close tabs	40 cycles	no memory or process leak	OK	
Close 59 of 60 tabs		memory returned (5.9 GB to 0.8 GB RSS)	OK	
Double click on ×	3 runs	new empty tab every time	FAIL	B-27
Close a tab during a calculation	8 s calculation	worker thread raises	FAIL	C-07
Save to a read-only file		nothing saved, no message	FAIL	C-07, M-04

Case	Size	Result	Verdict	Note
Ctrl+S before a new tab has loaded		nothing saved, no message	**FAIL**	C-07
Open the same file twice		two tabs, one path	**FAIL**	B-29
Close a saved tab, close Carbon		asks anyway	**FAIL**	U-01
Close Carbon with a frozen tab		still open after 6 s	**FAIL**	M-04 (still open)

Network, settings and dialogs

Case	Size	Result	Verdict	Note
PubChem offline	Tegn	"Ikke fundet"	**FAIL**	M-07
PubChem behind a login page	Tegn / Find navn	nothing / exception	**FAIL**	M-07, C-07
PubChem empty reply	Tegn	exception	**FAIL**	C-07
Changelog offline / login page / not UTF-8	Nyheder	nothing / page shown as news / exception	**FAIL**	M-08
License activation, slow server	3 s	dialog frozen, 0 timer ticks	**FAIL**	U-04
pK table	300 entries	0.87 s per add	**SLOW**	P-09
Toggle switch	1 000 clicks	0.5 s, setting saved	**OK**	
Damaged pK table in the settings		Carbon doesn't start	**FAIL**	C-08
Install path with a space (`url()` in style sheets)		identical rendering	**OK**	suspected, ruled out

4 Still open from the interpreter audit

In the last full run on this code, all 71 strict xfails in `test_stress_interpreter.py`, `test_stress_gui.py` and `test_stress_regneark.py` still xfailed, so nothing in `Interpreter_Audit_Report.pdf` has changed except M-05. I re-checked C-02, M-04 and WIP-01 by hand today.

ID	Finding	Status	Today
C-01	Zooming into a plot freezes the sheet	**OPEN**	
C-02	A damaged .crbn stops Carbon at start-up	**OPEN**	re-checked: an empty file still raises `JSONDecodeError` in `load_regneark`
C-05	One unprintable result hides every result	**OPEN**	
C-06	"export" sheets run on the GUI thread	**OPEN**	
M-04	Close prompt waits for every tab; save errors are silent	**OPEN**	re-checked: one frozen tab kept Carbon open; a read-only save showed nothing. U-01's fix removes the wait.
M-05	No Ctrl+S	**FIXED**	Ctrl+S, "Gem som" and the unsaved dot work (6 tests)
P-03, P-04	Big sheets are slow; undo grows without limit	**OPEN**	`reportDirty` now also serialises the sheet on every key (+7 ms per key on a 500-line sheet)
R-01	Result rows can be edited	**OPEN**	
WIP-01	`addSection` can't be called	**OPEN**	see WIP-02

5 State of the test suite

The last full run on exactly this code (at the end of our previous session) gave **54 failed, 724 passed, 93 xfailed** in 258 s, the same failures as in the interpreter report. Only six failures touch the GUI: five out-of-date expectations in `test_gui_latex.py` (`calc_to_latex` now writes fractions, assignments now print their value), and `test_path.py::test_build_bundles_resource[Assets_Ketcher]`, which is M-06.

6 The new stress tests

`tests/test_stress_window.py` has 41 tests in the style of your other stress files. **9 pass** and guard what works today: hostile panel inputs, rapid navigation, drawer toggling, extreme window sizes, huge searches, many tabs, long tab names, panel dragging and process clean-up. **32 are strict xfails**, at least one per finding, with the ID in the reason. I ran every xfail once with `--runxfail` to make sure it fails for the right reason and not a test bug. When you fix something, its test XPASSes and fails, which is your signal to remove the marker.

The file runs in about 90 s and gave the same result twice in a row. Settings go to a temporary file, so your registry isn't touched. Nothing uses the network, and no dialog waits for a click. It opens every panel once, so expect about 1.6 GB of memory for a few seconds. Run it headless in PowerShell:

```
$env:QT_QPA_PLATFORM = "offscreen"
python -m pytest -q -rxX tests\test_stress_window.py
```

7 What already works well

- **Nothing crashed.** No click, key, resize or input in any of the stress runs crashed Carbon, hung it or leaked a process.
- **Tabs clean up after themselves.** Closing 59 of 60 tabs gave the memory back (5.9 GB to 0.8 GB summed RSS), and 40 open/close cycles left no trace.
- **The panels are robust.** About 60 hostile inputs (empty boxes, letters, commas, 1e400, a 1 200-letter DNA strand) didn't raise a single exception. The messages need work (E-04), but nothing breaks.
- **The window can't be resized into a broken state.** From 1 × 1 to 6 000 × 4 000 with the drawer open, the layout keeps a sensible minimum, and dragged panels stay on the page.
- **The tab bar scales.** 60 tabs and 2 000-character names get scroll buttons, "+" stays reachable and the window doesn't grow.
- **Rendering is sharp** at 150 % and 200 % scaling: the SVG icons, the page watermarks and the fonts all scale cleanly.
- **The unsaved dot and Ctrl+S work**, including undo back to the saved state, and the search handles 100 000 characters, emoji and right-to-left text instantly.

8 Suggested order of work

- **First (an afternoon):** P-06 (one flag), M-06 (one line in `build.spec`), B-26 (two help texts), B-27 (two lines), U-01 + U-02 (dirty flag and default button), C-08, and the one-liners R-12, R-13 and R-15.
- **Then error handling:** C-07 (catch errors in callbacks, a global handler and a log file), then M-07, M-08, U-04 and E-04, so that students see Danish messages instead of nothing.
- **Then the panel display:** R-10, R-11 and R-14 (one output widget that wraps), then U-03, B-28 and R-17.
- **Then speed:** P-07 (cache the element data, then a worker), P-08 (lazy imports and lazy Ketcher) and P-09.
- **Finally:** U-05, U-06, B-29, B-30, E-05, Q-02, and the long-term fix for P-06: fewer web views.

That's all of it. I-it's not like I want your app to be good or anything. I just can't stand sloppy buttons.